

Módulo 09 - Apêndices e Revisão Final

1. Mapa Mental Textual do Projeto

BabyPlays

- Produto
 - aluguel de brinquedos
 - kits de festa prontos
 - kits personalizáveis
 - operação com entrega, retirada e higienização
- Backend Django
 - `catalogo`
 - `Brinquedo`
 - `Categoria`
 - `ImagemBrinquedo`
 - `UnidadeBrinquedo`
 - `KitFesta`
 - `ItemKitFesta`
 - `ConfiguracaoKitPersonalizavel`
 - `RegraCategoriaKitPersonalizavel`
 - `pedidos`
 - `Carrinho`
 - `ItemCarrinho`
 - `Pedido`
 - `ItemPedido`
 - `Contrato`
 - `AceiteContrato`
 - `ReservaUnidade`
 - `clientes`
 - `Cliente`

- cadastro
- login
- logout
- `/me/`
- CSRF
- `entregas`
 - configuração de taxa
 - CEP
 - provider de rota
 - cálculo de taxa
- API DRF
 - catálogo público
 - endpoints admin protegidos
 - carrinho por sessão
 - pedidos do próprio usuário
 - contrato e aceite
 - reserva de unidades
 - agenda admin
- Frontend Next.js
 - catálogo público
 - autenticação
 - painel admin
 - listagem e detalhe de pedidos
 - ações operacionais
 - agenda semanal
- Regras críticas
 - cliente só acessa os próprios dados
 - admin usa rotas protegidas
 - preço e frete são calculados no backend
 - disponibilidade é calculada por unidade física e período
 - carrinho não reserva estoque
 - pedido não confirma sem contrato aceito

- retirada manda unidade para higienização
- liberação para disponível é manual

2. Perguntas de Entrevista Baseadas no BabyPlays

Django e Modelagem

1. Por que `UnidadeBrinquedo` é uma entidade separada de `Brinquedo` ?
2. Qual problema a constraint de uma imagem principal por brinquedo resolve?
3. Por que `Categoria` usa `slug` único?
4. Quando usar `on_delete=PROTECT` em vez de `CASCADE` neste projeto?
5. Por que `Pedido` salva snapshots de valores e dados do cliente?

Django REST Framework

1. Por que existem serializers públicos e admin para brinquedos?
2. Como `get_permissions()` muda a superfície de segurança do `BrinquedoViewSet` ?
3. Por que endpoints admin usam `IsAdminUser` ?
4. Como o serializer impede o frontend de forjar taxa ou total?
5. Como a API de agenda evita expor dados sensíveis do aceite?

Regras de Negócio

1. Qual diferença entre validar seleção de kit e validar disponibilidade?
2. Por que adicionar item ao carrinho não reserva estoque?
3. Como o sistema evita overbooking?
4. Por que retirada manda unidade para `HIGIENIZACAO` ?
5. Por que a liberação de unidade não altera reservas nem pedidos?

Testes

1. Que teste prova que brinquedo inativo não aparece no catálogo?
2. Que teste prova que cliente não acessa pedido de outro cliente?
3. Que teste prova que pedido sem contrato não confirma?
4. Que teste prova rollback quando falta estoque?
5. Que teste prova que dados financeiros enviados pelo frontend são ignorados?

Autenticação, Sessão e CSRF

1. Por que o projeto removeu JWT?
2. Qual relação entre `credentials: "include"` e Django session?
3. Por que CSRF é necessário em `POST`, `PATCH`, `PUT` e `DELETE` ?
4. Como o carrinho anônimo é preservado após login?
5. Por que staff pode logar sem perfil `Cliente` ?

Next.js e TypeScript

1. Qual arquivo centraliza chamadas HTTP no frontend?
2. Como os tipos TypeScript representam contratos dos serializers?
3. Por que o painel admin consulta `/api/admin/me/` ?
4. Como a tela de pedido admin decide quais botões de ação exibir?
5. Onde ainda falta integração completa entre carrinho visual e carrinho backend?

3. Pontos Fortes Para Portfólio

- Modelagem de estoque por unidade física, não por contador simples.
- Separação clara entre catálogo público e administração.
- Uso de Django REST Framework com permissões explícitas.
- Serviços de domínio para regras críticas de carrinho, pedido, contrato, reserva e operação.
- Testes cobrindo regras de negócio, permissões, segurança e regressões.
- Autenticação por sessão Django com CSRF em vez de JWT desnecessário.
- Proteção de cliente contra acesso a dados de outros clientes.
- Fluxo operacional real: reservar, confirmar, iniciar locação, registrar retirada e liberar unidade.
- Aceite de contrato com snapshot e auditoria.
- Cálculo de taxa no backend, bloqueando campos financeiros forjados.
- Integração Next.js/TypeScript com API real.
- Painel administrativo com listagem, detalhe, ações e agenda operacional.
- Correções reais registradas em commits de fix, mostrando maturidade incremental.

4. Débitos Técnicos e Melhorias Futuras

Estes pontos vêm do código e do histórico atual. Não são novas funcionalidades prometidas.

- Frete por bairro versus taxa por distância: `docs/PROJECT_CONTEXT.md` fala em frete por bairro, mas o código atual calcula por CEP/distância. Antes de produção, alinhar regra de produto e implementação.
- Rate limit de login: o checklist de segurança cita proteção contra força bruta, mas o código atual não mostra rate limit.
- Testes frontend: há muitos testes backend, mas não há suíte frontend equivalente para componentes, guards e fluxos admin.
- Carrinho frontend/backend: o commit `f960611` adiciona contador local; ainda vale revisar integração completa com `CarrinhoService`.
- Logs versionados: `frontend/next-dev.log` e `frontend/next-dev.err.log` entraram no histórico. Revisar higiene de arquivos gerados.
- E-mails e WhatsApp: contexto do projeto cita confirmação, lembrete e reserva via WhatsApp/e-mail; o material estudado não mostra implementação desses fluxos.
- Política LGPD: checklist cita privacidade, anonimização/exclusão e cuidado com dados sensíveis; ainda merece plano específico.
- Deploy: settings já suportam ambiente, banco e cookies seguros, mas produção ainda pede validação com `check --deploy`, HTTPS, backup e hardening de VPS.
- Observabilidade operacional: ações críticas existem, mas auditoria detalhada de eventos administrativos ainda pode evoluir.
- Agenda: endpoint é simples e útil; próximos passos podem incluir horários reais, responsáveis, filtros adicionais e impressão operacional.

5. Plano de Revisão de 14 Dias

Dia 1 - Linha do Tempo

Leia `README.md` da trilha e rode:

```
git log --reverse --date=short --pretty=format:"%h %ad %s"
```

Objetivo: memorizar a evolução geral do produto.

Dia 2 - Fundação Django

Estude os commits `7b184a4`, `f77b1cd` e `8b82a77`.

Foco: models, serializers, viewsets, routers e permissions.

Dia 3 - Testes de Catálogo

Estude `67aac45`, `70dc172` e `272da36`.

Foco: transformar regra pública/admin em teste.

Dia 4 - Estoque Físico

Estude `bae215b` e `9fdde91`.

Foco: diferença entre publicação no catálogo e disponibilidade de unidade.

Dia 5 - Categorias e Imagens

Estude `187e056` e `62bf4f2`.

Foco: taxonomia, upload seguro e API pública limpa.

Dia 6 - Kits

Estude `ada8783`, `5c393e5` e `93e1ac8`.

Foco: composição, elegibilidade e validação de seleção.

Dia 7 - Carrinho e Pedido

Estude `7ad2e9c`, `a524126` e `b2bcfbc`.

Foco: sessão anônima, snapshot e fronteira pública.

Dia 8 - Entrega

Estude `f1e8a6f`, `ed5c440` e `2c50443`.

Foco: provider externo, cálculo backend e bloqueio de valores forjados.

Dia 9 - Contrato

Estude `13464c8`.

Foco: contrato vigente, aceite, snapshot e auditoria.

Dia 10 - Disponibilidade e Reserva

Estude `5ca95fe`, `179ff89` e `aa5286e`.

Foco: período, conflito, transação, rollback e overbooking.

Dia 11 - Operação

Estude `2ed40c5`, `9040731`, `c769c2a` e `33f8c10`.

Foco: confirmação, locação, retirada, higienização e ações disponíveis.

Dia 12 - Autenticação

Estude `d5bc892`, `a098e76`, `a0e6c5b`, `eeb170a`, `b4ba3c7`, `059c913` e `a9be2e2`.

Foco: sessão, CSRF, cliente, staff e remoção de JWT.

Dia 13 - Frontend

Estude `3f2019f`, `b9a9a92`, `e4c32d1`, `7d2d755`, `f960611` e `ebc1814`.

Foco: Next.js, TypeScript, cliente HTTP, sessão e contrato de API.

Dia 14 - Admin e Agenda

Estude `d262675`, `1b5a814`, `8f0b85a`, `b60d932`, `74b69c7`, `dba6094` e `02a9e10`.

Foco: painel admin, ações operacionais, agenda e segurança staff.

Revisão Final

Ao final dos 14 dias, explique sem consultar:

- o fluxo completo de um brinquedo do catálogo até a retirada;
- o fluxo completo de um cliente do carrinho anônimo ao pedido;
- por que contrato e reserva são etapas separadas;
- por que a unidade devolvida não volta automaticamente ao estoque;
- como frontend, sessão Django e CSRF trabalham juntos.